Vector DB (Search) Technical Specification v3.5

Decision-Oriented | Hardware-Aware | Evolutionary Narrative
Authors: Data Motifs, Gemini, ChatGPT

---

0. Notation Table

Symbol	Meaning
N	Number of stored vectors
D	Vector dimensionality
B	Bytes per element
n	Number of vectors touched (algorithmic skipping)
n_probe	Number of vectors probed in IVF cluster
D_eff	SIMD-aligned effective dimension
B_quant	Bytes per element after quantization
Γ	Locality multiplier (hardware)
γ_NUMA	NUMA factor (cross-socket memory)
Λ	Software / glue code overhead
Parallelism	Number of CPU cores / threads
Φ	Unified system strain
Φ_distributed	Distributed system strain
InstrCost	CPU instruction cost per operation (function of quantization)
K	Number of top results requested
S	Number of shards / nodes
χ	Contention factor (<1) for concurrent queries

D_eff — SIMD-aligned effective dimension. Usually rounds up to nearest SIMD lane multiple.
Example: If D = 128 → D_eff = 128; if D = 100 → D_eff = 112 or 128 depending on SIMD width (AVX, AVX512, etc.).

---

1. Historical Foundation: Unit Cost of Search

- **Computational Cost:**

$$C_{Search} = N \cdot D \cdot MathCost$$

- **Memory Transfer Cost:**

$$M_{Total} = N \cdot D \cdot B + Metadata$$

Observation:

- Naive linear search (K-NN) is O(N), memory-bound → Memory Wall.
- All later equations refine these fundamental costs.

---

2. Evolutionary Derivation: Algorithmic Costs

2.1 Naive Linear Search

$$C_{Unit} = D \cdot MathCost + D \cdot B \cdot MemoryAccessCost$$
$$C_{Total} = N \cdot C_{Unit}$$

- Linear in N, dominated by memory for large datasets.

2.2 Clustered Indexing (IVF)

$$C_{Total} = k \cdot C_{Unit} + n_{probe} \cdot \frac{N}{k} \cdot C_{Unit}$$

- k = number of clusters
- n_probe = number of clusters searched (tune for recall α)
- Reduces effective N → n, adds centroid search overhead.

2.3 Graph-Based Search (HNSW)

$$C_{HNSW} \approx efSearch \cdot \log(N) \cdot C_{Unit}$$

- efSearch = number of entry points explored (high recall α → higher efSearch).
- Visits only relevant neighbors; logarithmic cost with constant factor dependent on efSearch.

2.4 Algorithmic Skipping & Clustering

- Skipping factor: n ≪ N
- Partition vectors into clusters (IVF / HNSW)
- Efficiency Ratio (ER):

$$ER = \frac{DataUsed}{DataMoved}$$

- Reduces search cost from O(N) → O(√N) or O(log N).

2.5 Representation Evolution: Quantization

Method	Bytes / Dim	Effect
SQ	1	4× memory savings, minor accuracy loss
PQ	0.02–0.1	50× memory savings, fits billion-scale datasets
BQ	0.125	Ultra-fast XOR math; high throughput (512-bit / cycle)

- SQ: reduces B
- PQ: splits D → D_sub, reduces memory footprint
- BQ: compresses both B & D, speeds computation
- Instruction cost varies with quantization:

$$InstrCost = f(B_{quant})$$

---

3. Hardware & Software Awareness

3.1 Locality Multiplier (Γ)

Access Pattern	Γ Range	Notes
Sequential / Flat Scan	1.0	Prefetching effective
Clustered (IVF)	2–5	Sequential within clusters; some cache misses
Random / Graph (HNSW)	5–10	Pointer-chasing; CPU stalls
NUMA-crossing	+1–2×	γ_NUMA: cross-socket memory increases effective Γ

3.2 Glue Code & Software Overhead (Λ)

Source	Latency / vector	Notes
Serialization	1–10 μs	Memory mapping helps
Context Switching	5–50 μs	Pin threads to reduce OS hops
Network / RPC	50–500 μs	Cluster-dependent; consider percentile latency

Observation:

- Λ dominates small datasets (N < 10k).
- Total overhead scales linearly with vectors touched.

3.3 Unified System Strain (Φ)

Refined v3.5 Equation:

$$\Phi = \Lambda + \frac{\Gamma \cdot \gamma_{NUMA} \cdot (n \cdot D_{eff} \cdot B_{quant} \cdot InstrCost(B_{quant}))}{Parallelism \cdot \chi}$$

Where:

- n = vectors touched (skipping applied)
- D_eff = SIMD-aligned dimensions
- B_quant = bytes per element after quantization
- InstrCost(B_quant) = instruction cost per operation, depends on quantization
- Parallelism = CPU cores / threads
- γ_NUMA = NUMA locality factor
- χ < 1 = concurrency degradation

Interpretation:

- Λ → software/network latency
- Γ · γ_NUMA → hardware memory penalty
- χ → throughput reduction under concurrent queries
- Fraction → parallelized vector computation
- Φ → practical metric for real-world deployment

InstrCost — CPU instruction cost per operation (Bitwise > Integer > Float).
For **Binary Quantization (BQ)**, InstrCost ≈ **0.02× float32**, because XOR + POPCNT can process **512 bits per cycle** on modern CPUs.
This explicitly separates **compute throughput gains** from memory savings.

---

4. Distributed / Multi-Node Systems

$$\Phi_{Distributed} = \frac{\Phi_{Local}}{S} + (\Lambda_{Network} + Cost_{Merge}(K, S))$$

- Merge cost: O(K log S) (Top-K aggregation)
- Λ_Network = network latency
- Multi-node tax arises once N $\gtrsim$ 100M

Scaling Simulation (100M vectors):

Nodes (S)	RAM / Node	Latency Local	Latency Merge	Total
1	400 GB	500 ms	0 ms	500 ms
10	40 GB	50 ms	5 ms	55 ms
50	8 GB	10 ms	15 ms	25 ms

- Adding nodes beyond threshold increases merge/network latency.
- Profile for γ_NUMA and χ under concurrent load.

---

5. Practical Decision Table

Scale (N)	Bottleneck	Algorithm / Index	B / D	Γ	Λ	Notes
<10k	Software	Flat / In-Process	float32	1.0	Dominant	Flat scan faster; minimize network
10k–1M	Cache	IVF / Flat-GPU	int8	2–5	Moderate	Sequential cluster access improves Γ
1M–10M	Search n	HNSW	PQ	5–10	Minor	Algorithmic skipping critical; tune efSearch
10M–100M	RAM / Network	Disk-based IVF + Sharding	BQ	5–10	Significant	Monitor merge/network; γ_NUMA may spike
>100M	RAM & Network	Sharded HNSW + PQ/BQ	PQ/BQ	5–10	High	Plan Φ_distributed; consider χ; hierarchical merge & caching

---

6. Verification Checklist

- SIMD alignment: D_eff aligned to 16/32 (AVX2/AVX512)
- Memory bandwidth: Data size / latency ≤ RAM bandwidth
- Cache fit: Index vs L3 cache
- Glue check: Λ not dominating small N
- Quantization: B_quant matches memory/accuracy trade-offs
- Parallelism & contention: cores/threads sufficient; tune χ
- Multi-node: validate Φ_distributed, merge/network, NUMA effects

---

7. Cost Modeling

$$Monthly\_Cost = N \cdot D_{eff} \cdot B_{quant} \cdot GB\_Rate$$

- Memory dominates → prioritize reducing B_quant

---

8. Decision Flow

1. Input: N, D, B, Target Latency
2. Compute D_eff, size
3. Compute Φ_min = Size / Bandwidth
4. Check N < 15k? → Flat / In-Process
5. Select index based on α (Recall)
6. Adjust n, B_quant, Γ, Parallelism, χ
7. Compute Φ and verify ≤ Target
8. Deploy Locally or Distributed (Φ_distributed)

---

9. Evolutionary Summary

- Unit Cost → Memory Wall
- Algorithmic Skipping → reduce effective N
- Quantization → reduce D × B
- Hardware Awareness → Γ & γ_NUMA
- Software Awareness → Λ
- Unified System Strain → Φ combines all variables
- Cluster / Shard Extension → Φ_distributed

---

10. Notes / Recommendations

- Always simulate Φ before deployment (choose shard count, quantization levels)
- Datasets >10M → PQ or BQ essential
- Control Λ for small N; avoid unnecessary network layers
- Hardware tuning (SIMD, cache, RAM bandwidth) directly impacts Φ
- Document N, D, B, α, resulting Φ

---

11. Minor Caveats / Considerations

- Γ ranges for HNSW (5–10) are empirical; profile for your CPU & RAM
- Λ dominance less significant for fully optimized C++ or GPU in-process searches
- Distributed merge O(K log S) is naive; pre-aggregation can reduce cost
- Parameter tuning (n_probe, k, efSearch, quantization) critical for Φ
- Edge cases: small D vectors, GPU / mixed setups not modeled in Φ
- Network variability & memory bandwidth saturation can cause nonlinear effects

---

12. Decision Flow (Sanity Check)

Add a bandwidth saturation check to ensure realistic Φ predictions under high query rates:

$$\text{If } (n \cdot D_{eff} \cdot B_{quant} \cdot QPS) > \text{Total RAM Bandwidth} \implies \chi \downarrow$$

- **QPS** = Queries per second
- **χ** = Contention factor, reduces effective parallelism when memory bandwidth is saturated
- Ensures multi-query contention is captured before deployment

Decision Flow:

1. Input: N, D, B, Target Latency, QPS
2. Compute D_eff, Size
3. Compute Φ_min = Size / Bandwidth
4. N < 15k? → Flat / In-Process
5. Pick index & quantization (target α / recall)
6. Adjust n, B_quant, Γ, Parallelism, χ
7. **Sanity check:** bandwidth vs QPS → adjust χ
8. Compute Φ (or Φ_distributed)
9. Deploy locally or distributed

---

13. Visualizing Trade-offs (Recommended for Stakeholders)

- Create **"Recall vs Latency" frontier plots**:
  - X-axis: Latency (ms)
  - Y-axis: Recall (α)
  - Lines/curves for Flat, IVF, HNSW, PQ, BQ
- Demonstrates why we move from flat scanning → clustered → graph-based → quantized
- Makes **trade-offs between accuracy, memory footprint, and compute visible** to non-technical stakeholders